

件副本, 利用恢复系统故障的方法(即 REDO+UNDO), 才能将数据库恢复到一致性状态。

② 装入相应的日志文件副本(转储结束时刻的日志文件副本), 重做已完成的事务。首先扫描日志文件, 找出故障发生时已提交的事务的标识, 将其记入重做队列, 然后正向扫描日志文件, 对重做队列中的所有事务进行重做处理, 即将日志记录中“更新后的值”写入数据库。

这样就可以将数据库恢复至故障前某一时刻的一致性状态了。

介质故障的恢复需要数据库管理员介入, 但数据库管理员只需要重装最新转储的数据库副本和有关的各日志文件副本, 然后执行系统提供的恢复命令即可, 具体的恢复操作仍由数据库管理系统完成。

11.6 具有检查点的恢复技术

利用日志技术进行数据库恢复时, 恢复子系统必须搜索日志, 确定哪些事务需要重做, 哪些事务需要撤销。一般来说, 需要检查所有日志记录。但这样做有两个问题, 一是搜索整个日志将耗费大量的时间; 二是很多需要重做处理的事务实际上已将其更新操作结果写到了数据库中, 然而恢复子系统又重新执行了这些操作, 浪费了大量时间。为了解决这些问题, 又发展了具有检查点的恢复技术。这种技术在日志文件中增加一类新的记录——检查点(checkpoint)记录, 增加一个重新开始文件, 并让恢复子系统在登录日志文件期间动态地维护日志。

1. 检查点记录

检查点记录的内容包括:

- ① 建立检查点时刻所有正在执行的事务清单。
- ② 这些事务最近一个日志记录的地址。

2. 重新开始文件

重新开始文件用来记录各个检查点记录在日志文件中的地址。图 11.3 说明了建立检查点 C_i 时对应的日志文件和重新开始文件。

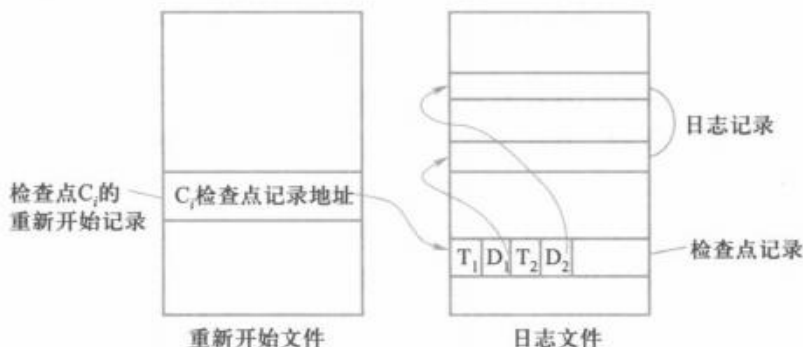


图 11.3 具有检查点的日志文件和重新开始文件

3. 动态维护日志文件

动态维护日志文件的方法是周期性地执行建立检查点、保存数据库状态的操作。具体步骤如下：

- ① 将当前日志缓冲区中的所有日志记录写入磁盘的日志文件中。
- ② 在日志文件中写入一个检查点记录。
- ③ 将当前数据缓冲区的所有数据记录写入磁盘的数据库中。
- ④ 把检查点记录在日志文件中的地址写入一个重新开始文件。

4. 恢复子系统的恢复策略

恢复子系统可以定期或不定期地建立检查点，保存数据库状态。检查点可以按照预定的一个时间间隔建立，如每隔一小时建立一个检查点；也可以按照某种规则建立检查点，如日志文件已写满一半时建立一个检查点。

使用检查点方法可以改善恢复效率。当事务 T 在一个检查点之前提交， T 对数据库所做的修改一定都已写入数据库，写入时间是在这个检查点建立之前或在这个检查点建立之时。这样，在进行恢复处理时就不必再对事务 T 执行重做操作了。

系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略，如图 11.4 所示。

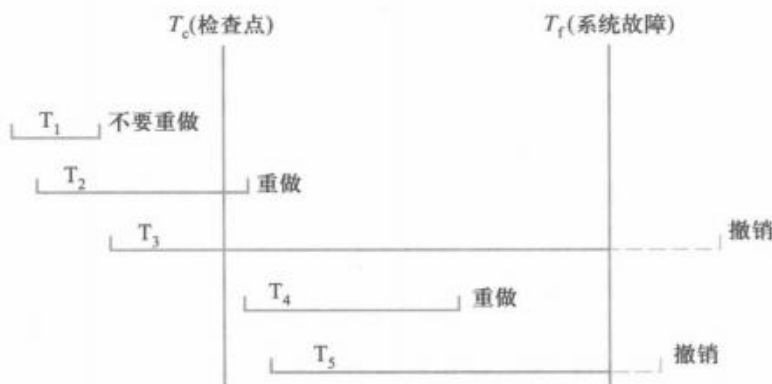


图 11.4 恢复子系统采取的不同策略

T_1 ：在检查点之前提交。

T_2 ：在检查点之前开始执行，在检查点之后故障点之前提交。

T_3 ：在检查点之前开始执行，在故障点时还未完成。

T_4 ：在检查点之后开始执行，在故障点之前提交。

T_5 ：在检查点之后开始执行，在故障点时还未完成。

T_3 和 T_5 在故障发生时还未完成，所以予以撤销； T_2 和 T_4 在检查点之后才提交，它们对数据库所做的修改在故障发生时可能还在缓冲区中，尚未写入数据库，所以要重做； T_1 在检查点之前已提交，所以不必执行重做操作。

系统使用检查点方法进行恢复的步骤是：

① 从重新开始文件中找到最后一个检查点记录在日志文件中的地址, 由该地址在日志文件中找到最后一个检查点记录。

② 由该检查点记录得到检查点建立时刻所有正在执行的事务清单 ACTIVE-LIST。

这里建立两个事务队列: UNDO-LIST 队列包含需要执行 UNDO 操作的事务集合, REDO-LIST 队列包含需要执行 REDO 操作的事务集合。

把 ACTIVE-LIST 暂时放入 UNDO-LIST 队列, REDO 队列暂为空。

③ 从检查点开始正向扫描日志文件。

a. 如有新开始的事务 T_i , 把 T_i 暂时放入 UNDO-LIST 队列。

b. 如有已提交的事务 T_j , 把 T_j 从 UNDO-LIST 队列移到 REDO-LIST 队列, 直到日志文件结束。

④ 对 UNDO-LIST 中的每个事务执行 UNDO 操作, 对 REDO-LIST 中的每个事务执行 REDO 操作。REDO 操作的起始点可以是 T_e 时刻。

11.7 数据库镜像

如前所述, 介质故障是对系统影响最为严重的一种故障。系统出现介质故障后, 用户应用全部中断, 恢复起来也比较费时。而且数据库管理员必须周期性地转储数据库, 这也加重了数据库管理员的负担。如果不及时正确地转储数据库, 一旦发生介质故障, 将会造成较大的损失。

随着技术的发展, 磁盘容量越来越大, 价格也越来越便宜。为避免磁盘介质出现故障影响数据库的可用性, 许多数据库管理系统提供了数据库镜像(mirror)功能用于数据库恢复。即根据数据库管理员的要求, 自动把整个数据库或其中的关键数据复制到另一个磁盘上, 每当主数据库更新时, 数据库管理系统自动把更新后的数据复制过去, 由数据库管理系统自动保证镜像数据与主数据库的一致性, 如图 11.5(a)所示。这样, 一旦出现介质故障, 可由镜像磁盘继续使用, 同时数据库管理系统自动利用镜像磁盘数据进行数据库的恢复, 不需要关闭系统和重装数据库副本, 如图 11.5(b)所示。在没有出现故障时, 数据库镜像还可以用于并发操作, 即当一个用户对数据加排他型锁修改数据时, 其他用户可以读镜像数据库上的数据, 而不必等待该用户释放锁。

由于数据库镜像是通过复制数据实现的, 频繁地复制数据自然会降低系统运行效率, 因此, 在实际应用中用户往往只选择对关键数据和日志文件进行镜像, 而不是对整个数据库进行镜像。

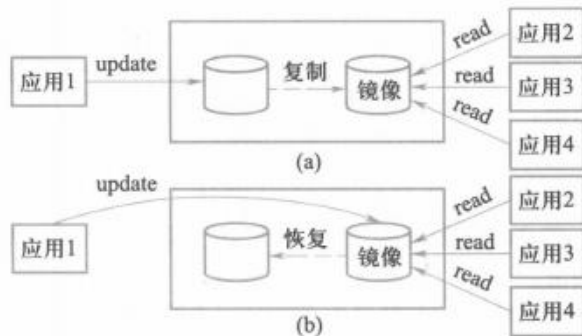


图 11.5 数据库镜像示意图